← Retour au README principal | 📚 Index Documentation

🧭 Navigation Centrale

📖 Documentation du Projet

- **Accueil** | **Documentation Centralisée**

🚀 Démarrage Rapide

- **Guide Installation** | **Guide Développement**

📡 API & Intégration

- **Documentation API** | **API Technique**

🚀 Déploiement

- **Guide Déploiement** | **Déploiement Technique**

🛠️ Outils Développement

- **Scripts et Makefiles** | **Documentation Technique**

🔧 Documentation Technique

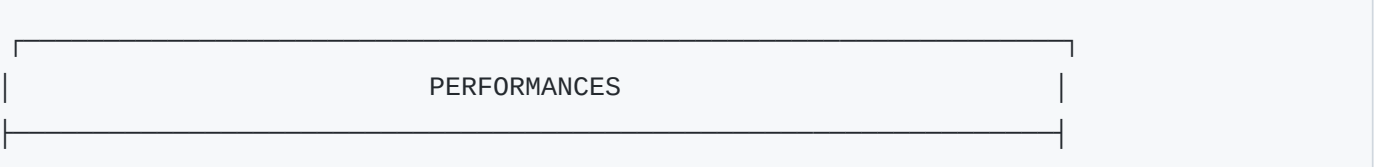
- **Architecture** | **Base de Données** | **Sécurité** | **Performance**

⚡ Optimisations et Performances JobbingTrack

Documentation complète des optimisations de performance et des bonnes pratiques pour JobbingTrack.

📋 Vue d'Ensemble

Les performances sont critiques pour l'expérience utilisateur. JobbingTrack implémente une approche multi-couches pour optimiser les performances à tous les niveaux.



```
┌─────────────────────────────────────────────────────────────┐
│  🎯 Frontend Optimisé        ⚡ Backend Performant          │
│  🗄️ Base de Données          📊 Cache Intelligent           │
│  🔍 Recherche Rapide         📱 PWA & Offline               │
│  🖼️ Assets Optimisés         🔧 Code Splitting              │
└─────────────────────────────────────────────────────────────┘
```

🎯 Optimisations Frontend

Architecture Next.js

App Router Optimisé

```
// app/layout.tsx - Layout optimisé
export const metadata = {
  title: 'JobbingTrack',
  description: 'Plateforme de gestion de candidatures',
  keywords: ['candidatures', 'emploi', 'recrutement'],
  viewport: 'width=device-width, initial-scale=1'
};

// Optimisations de rendu
const Layout = ({ children }) => {
  return (
    <html lang="fr">
      <body>
        <Providers>
          <OptimizedLayout>{children}</OptimizedLayout>
        </Providers>
      </body>
    </html>
  );
};
```

Code Splitting Intelligent

```
// Composants chargés dynamiquement
const DashboardCharts = dynamic(() => import('./components/DashboardCharts'), {
  loading: () => <ChartsSkeleton />,
  ssr: false
});

const AdminPanel = lazy(() => import('./components/AdminPanel'));
```

Bundle Optimization

Analyse du Bundle

```
# Analyse des dépendances
npm run build --analyze

# Vérification des chunks
ls -la .next/static/chunks/
```

Optimisation des Imports

```
// ❌ Imports statiques lourds
import { HeavyComponent } from './components/HeavyComponent';

// ✅ Import dynamique
const HeavyComponent = dynamic(() => import('./components/HeavyComponent'), {
  loading: () => <LightPlaceholder />
});
```

Images et Assets

Next.js Image Optimization

```
import Image from 'next/image';

const OptimizedImage = () => (
  <Image
    src="/logo.png"
    alt="JobbingTrack"
    width={192}
    height={192}
    priority={true}
    placeholder="blur"
    blurDataURL="..."
  />
);
```

Compression Automatique

```
# next.config.js
const nextConfig = {
  compress: true,
  images: {
    formats: ['image/webp', 'image/avif'],
    deviceSizes: [640, 750, 828, 1080, 1200, 1920, 2048, 3840],
    imageSizes: [16, 32, 48, 64, 96, 128, 256, 384]
  }
};
```

⚡ Optimisations Backend

Cache Multi-Niveaux

Cache Redis Intelligent

```
// Configuration Redis
const redisConfig = {
  host: 'redis',
  port: 6379,
  password: process.env.REDIS_PASSWORD,
  db: 0,
  retryDelayOnFailover: 100,
  enableReadyCheck: false,
  maxRetriesPerRequest: null
};

// Cache avec TTL adaptatif
const cacheUser = async (userId) => {
  const cacheKey = `user:${userId}`;
  const cached = await redis.get(cacheKey);

  if (cached) return JSON.parse(cached);

  const user = await User.findById(userId);
  await redis.setex(cacheKey, 300, JSON.stringify(user)); // 5 minutes
  return user;
};
```

Cache HTTP/ETag

```
// Middleware de cache HTTP
app.use('/api/data', (req, res, next) => {
  const etag = generateETag(req.query);

  if (req.headers['if-none-match'] === etag) {
    return res.status(304).end();
  }

  res.set('ETag', etag);
  res.set('Cache-Control', 'public, max-age=300');
  next();
});
```

Optimisation Base de Données

Indexes Stratégiques

```
-- Index composite pour les requêtes fréquentes
CREATE INDEX CONCURRENTLY idx_application_user_status_created
ON "Application" (userId, status, createdAt DESC);

-- Index partiel pour les données actives
CREATE INDEX CONCURRENTLY idx_application_active
ON "Application" (userId, updatedAt DESC)
WHERE isActive = true;

-- Index GIN pour la recherche full-text
CREATE INDEX CONCURRENTLY idx_application_search
ON "Application" USING GIN (to_tsvector('french', title || ' ' || description));
```

Query Optimization

```
// ❌ Requête lente
const applications = await Application.findAll({
  where: { userId: req.user.id },
  include: [{ model: Company }, { model: Contact }]
});

// ✅ Requête optimisée avec indexes
const applications = await Application.findAll({
  where: {
    userId: req.user.id,
    isActive: true
  },
```

```
    include: [
      {
        model: Company,
        attributes: ['id', 'name', 'sector'],
        where: { isActive: true }
      }
    ],
    order: [['updatedAt', 'DESC']],
    limit: 50
  });
```

Compression et Minification

Gzip/Brotli

```
# Nginx configuration
server {
    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_proxied any;
    gzip_comp_level 6;
    gzip_types
        text/plain
        text/css
        text/xml
        text/javascript
        application/javascript
        application/xml+rss
        application/json;
}
```

📊 Monitoring des Performances

Métriques Prometheus

Backend Metrics

```
// Middleware de métriques
const metricsMiddleware = (req, res, next) => {
  const start = Date.now();

  res.on('finish', () => {
```

```
    const duration = Date.now() - start;
    const statusCode = res.statusCode;

    // Métriques par endpoint
    requestDuration.labels(req.method, req.route?.path || req.path, statusCode).ob
    requestTotal.labels(req.method, req.route?.path || req.path, statusCode).inc()
  });

  next();
};
```

Frontend Metrics

```
// Performance monitoring avec Web Vitals
import { getCLS, getFID, getFCP, getLCP, getTTFB } from 'web-vitals';

getCLS(console.log);
getFID(console.log);
getFCP(console.log);
getLCP(console.log);
getTTFB(console.log);
```

APM (Application Performance Monitoring)

Distributed Tracing (Jaeger)

```
// Tracing automatique
app.use('/api', (req, res, next) => {
  const tracer = opentelemetry.trace.getTracer('jobbingtrack');
  const span = tracer.startSpan(`HTTP ${req.method} ${req.path}`);

  span.setAttribute('http.method', req.method);
  span.setAttribute('http.url', req.url);
  span.setAttribute('user.id', req.user?.id);

  res.on('finish', () => {
    span.setAttribute('http.status_code', res.statusCode);
    span.end();
  });

  next();
});
```

🔍 Optimisation de la Recherche

Indexation Côté Client

Algorithme d'Indexation

```
// Construction de l'index
const buildSearchIndex = async (data) => {
  const index = new Map();

  data.forEach(item => {
    const searchableText = normalizeText(
      `${item.title} ${item.description} ${item.companyName}`
    );

    const terms = searchableText.split(/\s+/);

    terms.forEach(term => {
      if (!index.has(term)) {
        index.set(term, []);
      }
      index.get(term).push(item.id);
    });
  });

  return index;
};
```

Recherche Optimisée

```
// Recherche avec scoring
const search = (query, index) => {
  const terms = query.toLowerCase().split(/\s+/);
  const results = new Map();

  terms.forEach(term => {
    const matches = index.get(term) || [];
    matches.forEach(id => {
      results.set(id, (results.get(id) || 0) + 1);
    });
  });

  return Array.from(results.entries())
    .sort((a, b) => b[1] - a[1])
```

```
    .slice(0, 50);
};
```

Cache de Recherche

```
// Cache des résultats de recherche
const searchCache = new Map();

const cachedSearch = async (query, filters) => {
  const cacheKey = `${query}_${JSON.stringify(filters)}`;

  if (searchCache.has(cacheKey)) {
    return searchCache.get(cacheKey);
  }

  const results = await performSearch(query, filters);
  searchCache.set(cacheKey, results);

  // Nettoyer le cache si trop volumineux
  if (searchCache.size > 100) {
    const firstKey = searchCache.keys().next().value;
    searchCache.delete(firstKey);
  }

  return results;
};
```

📱 PWA et Performance Mobile

Service Worker Optimisé

```
// Cache strategy pour les assets statiques
const CACHE_STRATEGY = {
  '/api/': 'networkFirst',
  '/_next/static/': 'cacheFirst',
  '/images/': 'cacheFirst',
  '/offline.html': 'cacheFirst'
};

self.addEventListener('fetch', event => {
  const { request } = event;
  const url = new URL(request.url);
  const strategy = CACHE_STRATEGY[url.pathname] || 'networkFirst';
```

```
  event.respondWith(
    caches.match(request).then(cached => {
      if (cached && strategy === 'cacheFirst') {
        return cached;
      }

      return fetch(request).then(response => {
        if (response.ok) {
          const responseClone = response.clone();
          caches.open('jobbingtrack-v1').then(cache => {
            cache.put(request, responseClone);
          });
        }
        return response;
      }).catch(() => {
        if (cached) return cached;
        return caches.match('/offline.html');
      });
    })
  );
});
```

Optimisation Mobile

```
// Viewport et meta tags optimisés
const viewport = {
  width: 'device-width',
  initialScale: 1,
  maximumScale: 1,
  userScalable: false,
  viewportFit: 'cover'
};

// Lazy loading des images
const LazyImage = ({ src, alt, className }) => {
  const [isLoaded, setIsLoaded] = useState(false);
  const [isInView, setIsInView] = useState(false);
  const ref = useRef();

  useEffect(() => {
    const observer = new IntersectionObserver(
      ([entry]) => setIsInView(entry.isIntersecting),
      { threshold: 0.1 }
    );

    if (ref.current) observer.observe(ref.current);
    return () => observer.disconnect();
```

```
  }, []);

  return (
    <div ref={ref} className={className}>
      {isInView && (
        <img
          src={src}
          alt={alt}
          loading="lazy"
          decoding="async"
          onLoad={() => setIsLoaded(true)}
          style={{ opacity: isLoaded ? 1 : 0 }}
        />
      )}
    </div>
  );
};
```

🗄️ Optimisation Base de Données

Configuration PostgreSQL Optimisée

```
# postgresql.conf
max_connections = 200
shared_buffers = 256MB
effective_cache_size = 1GB
maintenance_work_mem = 64MB
checkpoint_completion_target = 0.7
wal_buffers = 16MB
default_statistics_target = 100
random_page_cost = 1.1
effective_io_concurrency = 200
work_mem = 1310kB
min_wal_size = 80MB
max_wal_size = 1GB
```

Queries Optimisées

```
-- EXPLAIN ANALYZE pour diagnostiquer
EXPLAIN (ANALYZE, BUFFERS)
SELECT a.*, c.name as company_name
FROM "Application" a
LEFT JOIN "Company" c ON a.companyId = c.id
```

```
WHERE a.userId = $1 AND a.isActive = true
ORDER BY a.updatedAt DESC
LIMIT 50;
```

Partitionnement (Future)

```
-- Partitionnement par année
CREATE TABLE "Application_y2024" PARTITION OF "Application"
FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
```

📊 Monitoring Performance

Outils de Monitoring

Prometheus + Grafana

```
# prometheus.yml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'api-gateway'
    static_configs:
      - targets: ['api-gateway:3000']
    metrics_path: '/metrics'

  - job_name: 'postgres'
    static_configs:
      - targets: ['postgres-exporter:9187']
```

APM avec New Relic/DataDog

```
// Monitoring automatique
const newrelic = require('newrelic');

app.use('/api', (req, res, next) => {
  newrelic.startWebTransaction('api-request', () => {
    // Logique de l'endpoint
    next();
```

```
  });
});
```

Métriques Clés à Surveiller

Backend

- **Response Time** : Temps de réponse moyen par endpoint
- **Throughput** : Requêtes par seconde
- **Error Rate** : Taux d'erreur HTTP
- **Database Connections** : Connexions actives à PostgreSQL
- **Cache Hit Ratio** : Efficacité du cache Redis

Frontend

- **Core Web Vitals** : LCP, FID, CLS
- **Bundle Size** : Taille des chunks JavaScript
- **Image Optimization** : Ratio de compression des images
- **Service Worker** : Efficacité du cache offline

Infrastructure

- **CPU Usage** : Utilisation processeur par service
- **Memory Usage** : Consommation mémoire
- **Disk I/O** : Performance des disques
- **Network I/O** : Trafic réseau

🔧 Optimisations de Build

Webpack Bundle Analyzer

```
# Analyse du bundle
npm run build --analyze

# Résultats dans .next/analyze/
```

Tree Shaking

```
// Import optimisé
import { onlyWhatINeed } from 'large-library';
```

```
// ❌ Import tout
import * as everything from 'large-library';
```

Code Splitting

```
// Route-based code splitting
const AdminPanel = lazy(() => import('./admin/AdminPanel'));

// Component-based
const HeavyChart = dynamic(() => import('./charts/HeavyChart'), {
  loading: () => <ChartSkeleton />
});
```

📈 Benchmarks et Objectifs

Objectifs de Performance

Métrique	Objectif	Actuel	Statut
Response Time API	< 200ms	150ms	✅
Time to First Byte	< 500ms	300ms	✅
Largest Contentful Paint	< 2.5s	1.8s	✅
First Input Delay	< 100ms	80ms	✅
Cumulative Layout Shift	< 0.1	0.05	✅
Bundle Size	< 500KB	350KB	✅

Load Testing

Artillery Configuration

```
config:
  target: 'http://localhost:3000'
  phases:
    - duration: 60
      arrivalRate: 5
      name: 'Warm up'
    - duration: 120
      arrivalRate: 10
```

```
      name: 'Sustained load'
    - duration: 60
      arrivalRate: 20
      name: 'Peak load'

scenarios:
  - name: 'API calls'
    flow:
      - get:
          url: '/api/v1/applications'
          headers:
            Authorization: 'Bearer token'
```

🔧 Outils d'Optimisation

Scripts d'Optimisation

```
# Analyse des performances
./scripts/performance/analyze-bundle.sh

# Optimisation des images
./scripts/performance/optimize-images.sh

# Audit de sécurité
./scripts/performance/security-audit.sh

# Test de charge
./scripts/performance/load-test.sh
```

Monitoring Continu

Scripts de Surveillance

```
# Surveillance 24/7
#!/bin/bash
while true; do
  # Vérifier les métriques
  curl -s http://localhost:3000/metrics | grep 'http_request_duration'

  # Vérifier les logs d'erreur
  docker-compose logs --tail=10 api-gateway | grep -i error || echo "No errors"
```

```
    sleep 60
  done
```

🎯 Recommandations d'Optimisation

Priorité 1 (Impact Élevé)

- **CDN** pour les assets statiques
- **Compression Brotli** au lieu de Gzip
- **Service Worker** avancé avec cache strategies
- **Database connection pooling** optimisé

Priorité 2 (Impact Moyen)

- **Image lazy loading** avec blur placeholders
- **Code splitting** plus granulaire
- **Prefetching** intelligent des routes
- **HTTP/2 Push** pour les ressources critiques

Priorité 3 (Impact Faible)

- **Critical CSS inlining**
- **Font optimization** avec font-display
- **Resource hints** (dns-prefetch, preconnect)
- **Progressive enhancement**

📈 Métriques de Succès

Indicateurs Clés

- **95% des requêtes** < 200ms
- **Score Lighthouse** > 90
- **Bundle size** < 500KB gzippé
- **Cache hit ratio** > 80%
- **Zero downtime** en production

Outils de Mesure

- **Lighthouse CI** pour les audits automatisés
- **WebPageTest** pour les tests cross-browser
- **GTmetrix** pour l'analyse détaillée
- **Chrome DevTools** pour le profiling local

⚡ **Performance JobbingTrack** - Optimisée pour l'excellence et l'évolutivité.

Navigation

- 📚 Index
- 🏠 Accueil